C Programming – Beginner's Guide

A clear, simple, easy-to-understand introduction to C programming designed like a student notebook. These notes explain the basics in a friendly way so that even a complete beginner can follow along and build confidence while learning.

Table of Contents

---

1. Introduction to C

C is one of the most powerful and influential programming languages, created in 1972 by Dennis Ritchie. It's used to build:

- **Operating systems** (Linux, Windows, macOS kernels)
- **Embedded systems** (IoT devices, microcontrollers)
- **Compilers** (tools that translate code)
- **High-performance applications** (games, databases)

Why Learn C?

- **Foundation of modern languages** – C++, Java, JavaScript, and Python share C's syntax
- **Close to hardware** – understand how computers actually work
- **Memory control** – learn to manage resources efficiently

- **Career opportunities** – essential for system programming, embedded development

---

2. How C Programs Run (Compilation Steps)

When you write C code, the computer cannot run it directly. It must be **compiled** into machine code.

The Compilation Process:

```
.c files  ->  Preprocess  ->  Compile  ->  Linker  → Executable
```

Steps Explained:

1. **Preprocessing** – Expands `#include`, `#define` macros, removes comments
2. **Compilation** – Converts C code into assembly language
3. **Assembly** – Translates assembly into machine code (object files)
4. **Linking** – Combines object files and libraries into final executable

Compiling Your First Program:

```
# Using GCC compiler
gcc program.c -o program

# Run the program
./program          # On Linux/Mac
program.exe        # On Windows
```

---

3. Basic Syntax & Structure of a C File

Every C program follows this basic structure:

```c
#include <stdio.h>  // Include standard input/output library

int main() {
    printf("Hello, C!\n");
    return 0;
}
```

Breakdown:

- `#include <stdio.h>` → Adds standard input/output functions like `printf` and `scanf`
- `int main()` → Starting point of every C program (required)
- `printf()` → Outputs text to the screen
- `return 0;` → Tells the operating system the program ended successfully

Important Notes:

- **Semicolons (;)** end every statement
- **Curly braces {}** define code blocks
- **C is case-sensitive** – `Main` is different from `main`
- **Comments** help explain code:

```
// Single-line comment
/* Multi-line
   comment */
```

---

4. Variables & Data Types

Variables are named containers that store data in memory.

Common Data Types:

Type	Purpose	Size (typical)	Example
`int`	Whole numbers	4 bytes	`int age = 20;`
`float`	Decimal numbers	4 bytes	`float g = 9.8;`
`double`	Larger decimals	8 bytes	`double pi = 3.14159;`
`char`	Single character	1 byte	`char letter = 'A';`
`char[]`	String (text)	varies	`char name[] = "Ali";`

Variable Declaration:

```
int age;            // Declaration
age = 20;           // Assignment

int score = 100;    // Declaration + initialization

const float PI = 3.14159;  // Constant (cannot change)
```

Memory Visualization:

```
int age = 20;

  age       20

Memory address → Value
```

Modifiers:

- `unsigned` – Only positive values: `unsigned int count = 5;`
- `short` – Smaller range: `short int x = 100;`
- `long` – Larger range: `long int big = 1000000;`

---

5. Operators

Arithmetic Operators:

```
int a = 10, b = 3;
a + b;   // 13 (addition)
a - b;   // 7  (subtraction)
a * b;   // 30 (multiplication)
a / b;   // 3  (integer division)
a % b;   // 1  (modulo/remainder)
```

Comparison Operators:

```
==  // Equal to
!=  // Not equal to
>   // Greater than
<   // Less than
>=  // Greater than or equal
<=  // Less than or equal
```

Logical Operators:

```
&&  // AND (both must be true)
||  // OR (at least one true)
!   // NOT (inverts true/false)
```

Assignment Operators:

```
=    // Assign
+=   // Add and assign (x += 5 means x = x + 5)
-=   // Subtract and assign
*=   // Multiply and assign
/=   // Divide and assign
```

Increment/Decrement:

```
i++;  // Increment by 1 (post-increment)
++i;  // Increment by 1 (pre-increment)
i--;  // Decrement by 1
```

---

6. Input & Output

Printing Output (`printf`):

```
printf("Hello, World!\n");

int age = 20;
printf("Age: %d\n", age);          // %d for integers
printf("Pi: %.2f\n", 3.14159);     // %.2f for 2 decimal places
printf("Letter: %c\n", 'A');       // %c for characters
printf("Name: %s\n", "Alice");     // %s for strings
```

Common Format Specifiers:

- `%d` – integer
- `%f` – float/double
- `%c` – character
- `%s` – string
- `%p` – pointer address
- `%x` – hexadecimal

Reading Input (`scanf`):

```
int age;
printf("Enter your age: ");
scanf("%d", &age);  // Note the & (address-of operator)

char name[50];
printf("Enter name: ");
scanf("%s", name);  // No & needed for arrays
```

Important: Always use `&` with `scanf` for single variables (except arrays).

---

7. Control Flow

Control flow statements let your program make decisions and repeat actions. They control the **order** in which code executes.

---

if / else Statements

What it does: Makes decisions based on conditions (true/false)

When to use: When you need different actions based on different situations

```
int age = 18;

if (age >= 18) {
    printf("You can vote");
} else {
    printf("Too young to vote");
}
```

Real-world examples: - Checking if a password is correct - Validating user input - Grading system (A, B, C, D, F)

Multiple Conditions (else if):

```
int score = 85;

if (score >= 90) {
    printf("Grade: A");
} else if (score >= 80) {
    printf("Grade: B");
} else if (score >= 70) {
    printf("Grade: C");
} else {
    printf("Grade: F");
}
```

How it works: Checks conditions from top to bottom, executes the first true condition, then skips the rest.

---

switch Statement

What it does: Compares one value against multiple possible matches

When to use: When you have many exact value comparisons (like menu options)

```
int choice;
printf("1. Start\n2. Options\n3. Exit\nChoose: ");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Starting game...");
```

```
        break;
    case 2:
        printf("Opening options...");
        break;
    case 3:
        printf("Goodbye!");
        break;
    default:
        printf("Invalid choice");
}
```

When to prefer switch over if: - Menu systems - Day of week (1-7) - Command interpreters - State machines

Important: Always use `break` to prevent fall-through (executing multiple cases).

Fall-through example (intentional):

```
switch(day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        printf("Weekday");
        break;
    case 6:
    case 7:
        printf("Weekend");
        break;
}
```

Loops

Loops repeat code automatically, saving you from writing the same thing many times.

---

for Loop **What it does:** Repeats code a **specific number of times**

When to use: When you know exactly how many iterations you need

```
// Print numbers 0 to 4
for(int i = 0; i < 5; i++) {
    printf("%d ", i);  // Output: 0 1 2 3 4
}
```

Structure:

```
for(initialization; condition; increment) {
    // code to repeat
}
```

Real-world examples: - Processing each element in an array - Printing multiplication tables - Drawing patterns with stars - Counting down from 10

Example - Sum of numbers:

```
int sum = 0;
for(int i = 1; i <= 10; i++) {
    sum += i;  // sum = sum + i
}
printf("Sum: %d", sum);  // Output: Sum: 55
```

Example - Array processing:

```
int scores[5] = {85, 90, 78, 92, 88};
for(int i = 0; i < 5; i++) {
    printf("Score %d: %d\n", i+1, scores[i]);
}
```

---

while Loop **What it does:** Repeats code **as long as** a condition is true

When to use: When you don't know how many iterations you need in advance

```
int n = 5;
while(n > 0) {
    printf("%d ", n);  // Output: 5 4 3 2 1
    n--;
}
```

Real-world examples: - Reading until end of file - Game loops (while player is alive) - Input validation (keep asking until valid input) - Processing unknown amount of data

Example - Input validation:

```
int age;
printf("Enter age (1-120): ");
scanf("%d", &age);

while(age < 1 || age > 120) {
    printf("Invalid! Try again: ");
    scanf("%d", &age);
}
printf("Valid age: %d", age);
```

Example - Menu system:

```c
int choice = 0;
while(choice != 3) {
    printf("\n1. Play\n2. Settings\n3. Quit\n");
    scanf("%d", &choice);

    if(choice == 1) printf("Starting...\n");
    else if(choice == 2) printf("Settings...\n");
}
```

---

do-while Loop **What it does:** Executes code **at least once**, then repeats if condition is true

When to use: When you need to run code before checking the condition

```c
int x = 0;
do {
    printf("This runs at least once\n");
    x++;
} while(x < 0);  // Condition is false, but still ran once
```

Key difference from while: Checks condition **after** executing code block

Real-world examples: - Menus (show menu at least once) - Games (play at least one round) - ATM operations (do transaction then ask to continue)

Example - Menu that must show:

```c
int choice;
do {
    printf("\n=== MENU ===\n");
    printf("1. Add\n2. Subtract\n3. Exit\n");
    printf("Choice: ");
    scanf("%d", &choice);

    if(choice == 1) printf("Adding...\n");
    else if(choice == 2) printf("Subtracting...\n");

} while(choice != 3);
```

Loop Control Statements

break **What it does:** Immediately exits the current loop

When to use: When you find what you're looking for and don't need to continue

```c
// Find first even number
int nums[] = {1, 3, 5, 8, 9, 12};
for(int i = 0; i < 6; i++) {
    if(nums[i] % 2 == 0) {
        printf("First even: %d", nums[i]);
        break;  // Stop searching
    }
}
```

Real-world uses: - Search algorithms (stop when found) - Error detection (exit on first error) - Exit infinite loops when condition is met

---

continue **What it does:** Skips the rest of the current iteration and jumps to the next one

When to use: When you want to skip certain values but keep looping

```c
// Print only odd numbers
for(int i = 1; i <= 10; i++) {
    if(i % 2 == 0) {
        continue;  // Skip even numbers
    }
    printf("%d ", i);  // Output: 1 3 5 7 9
}
```

Real-world uses: - Skip invalid data - Process only certain elements - Filter results

---

Choosing the Right Control Flow

Scenario	Use This
Decision based on condition	`if/else`
Multiple exact value comparisons	`switch`
Know exact number of repetitions	`for` loop
Repeat until condition changes	`while` loop
Must execute at least once	`do-while` loop
Exit early from loop	`break`
Skip current iteration	`continue`

Nested Control Flow

You can put control structures inside each other:

```c
// Nested loops - print multiplication table
for(int i = 1; i <= 3; i++) {
    for(int j = 1; j <= 3; j++) {
        printf("%d x %d = %d\n", i, j, i*j);
    }
    printf("\n");
}
```

Common pattern - 2D arrays:

```c
int matrix[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}};

for(int row = 0; row < 3; row++) {
    for(int col = 0; col < 3; col++) {
        printf("%d ", matrix[row][col]);
    }
    printf("\n");
}
```

---

8. Functions

Functions break programs into reusable, manageable pieces.

Function Syntax:

```c
return_type function_name(parameters) {
    // code
    return value;
}
```

Example:

```c
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    printf("Sum: %d\n", result);  // Output: Sum: 7
    return 0;
}
```

Function Prototype:

```c
int add(int, int);  // Declaration (tells compiler function exists)
```

```c
int main() {
    printf("%d", add(5, 3));
    return 0;
}

int add(int a, int b) {  // Definition
    return a + b;
}
```

void Functions (no return value):

```c
void greet() {
    printf("Hello!\n");
}
```

---

9. Arrays

An array stores multiple values of the same type in contiguous memory.

Declaration:

```c
int nums[5];              // Declare array of 5 integers
int scores[3] = {10, 20, 30};  // Initialize with values
```

Memory Layout:

```
 10    20    30

  0     1     2    (indices)
```

Accessing Elements:

```c
printf("%d", scores[0]);  // 10 (first element)
scores[1] = 25;           // Modify second element
```

Looping Through Arrays:

```c
int nums[5] = {1, 2, 3, 4, 5};

for(int i = 0; i < 5; i++) {
    printf("%d ", nums[i]);
}
```

Multi-dimensional Arrays:

```c
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

printf("%d", matrix[0][1]);  // 2
```

---

10. Strings

A string is an array of characters ending with `\0` (null terminator).

Declaration:

```c
char name[] = "Ada";  // Compiler adds \0 automatically

// Memory: A  d  a  \0
```

String Functions (need `#include <string.h>`):

```c
strlen(str)        // Get length
strcpy(dest, src)  // Copy string
strcat(dest, src)  // Concatenate strings
strcmp(s1, s2)     // Compare strings
```

Example:

```c
#include <string.h>

char greeting[20] = "Hello";
strcat(greeting, " World");  // greeting = "Hello World"
printf("%s\n", greeting);
```

Reading Strings:

```c
char name[50];
scanf("%s", name);      // Reads until whitespace
fgets(name, 50, stdin); // Reads entire line
```

---

11. Pointers

A pointer stores the memory address of a variable.

Syntax:

```c
int x = 10;
int *p = &x;  // p points to x's address
```

Visualization:

```
x = 10

 10   at address 0x1000

 ↑

p = 0x1000 (stores address of x)
```

Dereferencing:

```c
printf("%d", *p);  // 10 (value at address)
*p = 20;           // Changes x to 20
```

Pointer Operators:

- `&` – Address-of operator
- `*` – Dereference operator

Pointers and Arrays:

```c
int arr[3] = {1, 2, 3};
int *p = arr;  // Array name is a pointer to first element

printf("%d", *p);      // 1
printf("%d", *(p+1));  // 2
```

NULL Pointer:

```c
int *p = NULL;  // Points to nothing (safe initialization)
```

---

12. Structures (struct)

A struct groups different data types into a single custom type.

Definition:

```c
struct Student {
    char name[50];
    int age;
```

```
    float gpa;
};
```

Usage:

```
struct Student s1;
strcpy(s1.name, "Ali");
s1.age = 16;
s1.gpa = 3.8;

printf("%s is %d years old\n", s1.name, s1.age);
```

Initialization:

```
struct Student s2 = {"Sara", 17, 3.9};
```

Pointers to Structs:

```
struct Student *sp = &s1;
printf("%d", sp->age);  // Use -> for pointer access
```

typedef (create alias):

```
typedef struct {
    char title[100];
    int pages;
} Book;

Book myBook = {"C Programming", 500};
```

---

13. Dynamic Memory (malloc, free)

Dynamic memory allocation lets you create variables at runtime.

Allocation:

```
#include <stdlib.h>

int *p = (int*)malloc(sizeof(int));  // Allocate memory for 1 integer
*p = 50;
```

Visualization:

```
Heap Memory:

 50   ← p points here
```

Allocating Arrays:

```c
int *arr = (int*)malloc(5 * sizeof(int));  // Array of 5 integers
arr[0] = 10;
arr[1] = 20;
```

Freeing Memory:

```c
free(arr);      // Release memory
arr = NULL;     // Good practice to avoid dangling pointers
```

calloc (initialized to zero):

```c
int *p = (int*)calloc(5, sizeof(int));  // 5 integers, all set to 0
```

realloc (resize memory):

```c
p = (int*)realloc(p, 10 * sizeof(int));  // Resize to 10 integers
```

Always free dynamically allocated memory to prevent memory leaks!

---

14. File Handling

File operations allow programs to save and read data permanently.

Opening a File:

```c
FILE *f = fopen("data.txt", "r");  // Open for reading

if (f == NULL) {
    printf("Error opening file\n");
    return 1;
}
```

File Modes:

- `"r"` – Read only
- `"w"` – Write (overwrites existing file)
- `"a"` – Append (adds to end)
- `"r+"` – Read and write

Reading from File:

```c
char line[100];
fgets(line, sizeof(line), f);  // Read one line
printf("%s", line);
```

Writing to File:

```c
FILE *f = fopen("output.txt", "w");
fprintf(f, "Hello, File!\n");
```

Closing File:

```c
fclose(f);  // Always close files when done
```

Example (Read entire file):

```c
FILE *f = fopen("data.txt", "r");
char ch;

while((ch = fgetc(f)) != EOF) {
    printf("%c", ch);
}

fclose(f);
```

---

15. Glossary of C Terms

Term	Definition
Variable	Named storage location in memory
Pointer	Variable that stores a memory address
Array	Collection of elements of the same type
String	Character array ending with `\0`
Struct	User-defined type grouping multiple variables
Heap	Dynamic memory area (malloc allocates here)
Stack	Automatic memory area (local variables)
Function	Reusable block of code
Compile	Convert source code to machine code
Preprocessor	Handles `#include`, `#define` before compilation
NULL	Pointer pointing to nothing (address 0)
malloc	Allocate dynamic memory
free	Release dynamically allocated memory
fopen	Open a file
fgets	Read a line from file

Term	Definition
EOF	End of file marker

---

16. Practice Exercises

(A) Variables & Data Types

```
// 1. Declare an integer for your age and print it
// 2. Create variables for float, char, and display them
```

(B) Pointers

```
// 1. Create int x = 15 and a pointer to it
// 2. Print value using the pointer
// 3. Change x's value through the pointer
```

(C) Arrays

```
// 1. Create int nums[3] = {2, 4, 6}
// 2. Print each value using a loop
// 3. Find sum of all elements
```

(D) File Handling

```
// 1. Create a text file "hello.txt" with some content
// 2. Write a program to read and print it
// 3. Write a program that counts lines in the file
```

---

17. Common Beginner Mistakes

1. Forgetting Semicolons

```
printf("Hello")
printf("Hello");
```

2. Array Index Out of Bounds

```
int arr[5];
arr[5] = 10;  //  Valid indices: 0-4
```

3. Not Using & with scanf

```c
int x;
scanf("%d", x);   //  Wrong
scanf("%d", &x);  //  Correct
```

4. Forgetting to Free Memory

```c
int *p = malloc(sizeof(int));
// ... use p ...
//  Forgot: free(p);
```

5. Uninitialized Variables

```c
int x;
printf("%d", x);  //  Undefined behavior (garbage value)
```

6. String Assignment

```c
char name[20];
name = "Alice";      //  Wrong
strcpy(name, "Alice"); //  Correct
```
